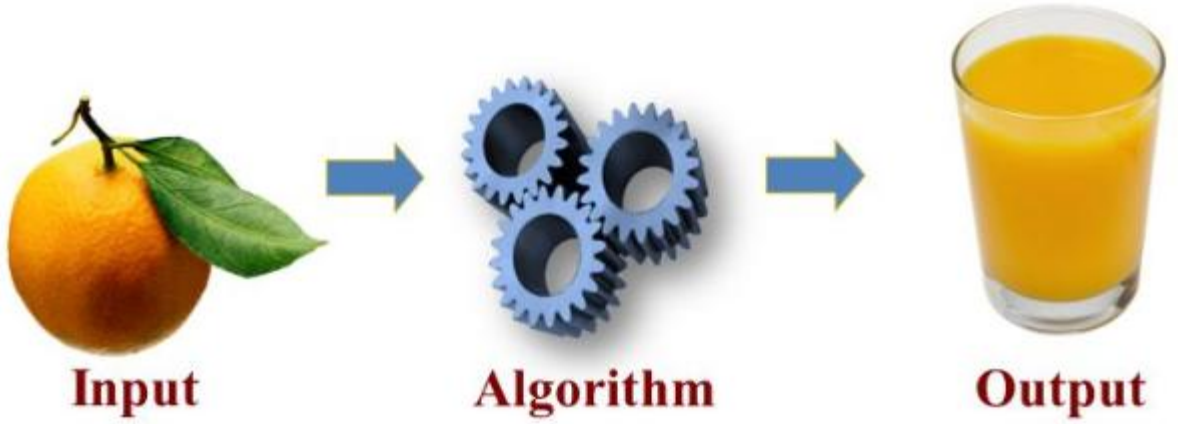
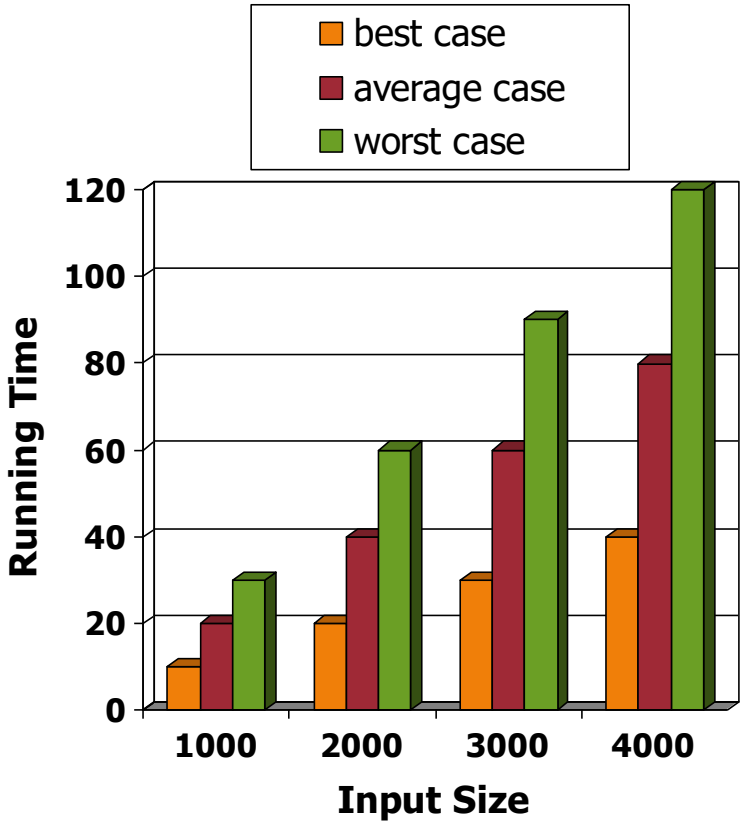
Lecture 01
Analysis of Algorithms



An **algorithm** is a step-by-step procedure for solving a problem in a finite amount of time.

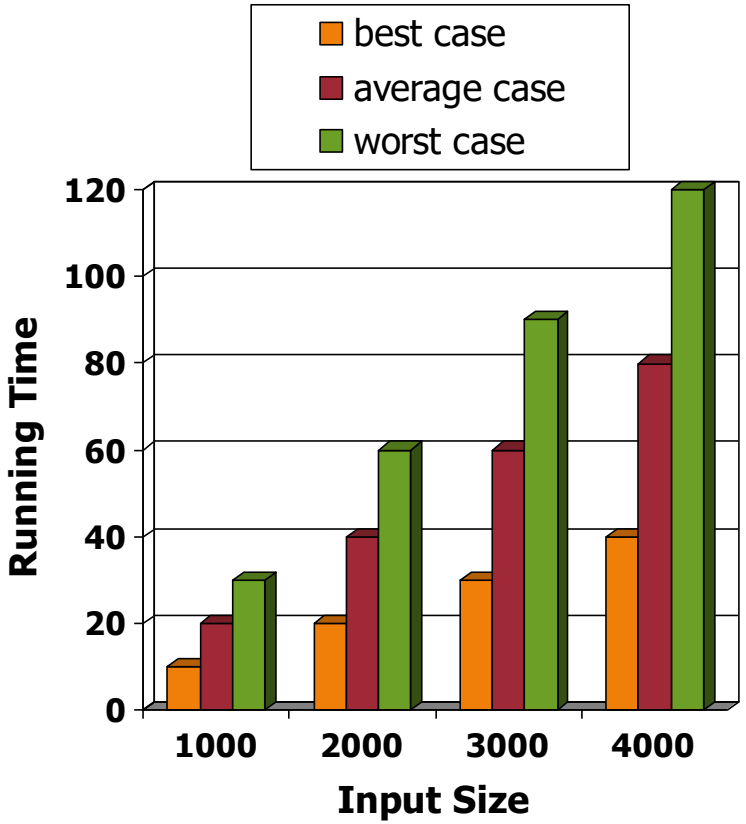
Running Time

- How well an algorithm performs when input size grows?
- The running time of an algorithm typically grows with the input size.
- An algorithm can perform differently even if the input size is the same but have a different contents or order.



Best, Worst, and Average Cases

- Given the same input size of n elements,
  - *Best case* occurs when an algorithm performs the minimum number of steps.
  - *Worst case* occurs when an algorithm performs the maximum number of steps.
  - *Average case* occurs when an algorithm performs the average number of steps.
- Average case time is often difficult to determine.



Examples of Best, Worst, and Average Cases

- Best case occurs when e exists as the first element in A, the 'if' runs once only.
- Worst case occurs when e does not exist in A, the 'if' runs n times.
- Average case occurs when e exists in the middle of A, the 'if' runs about n/2 times.

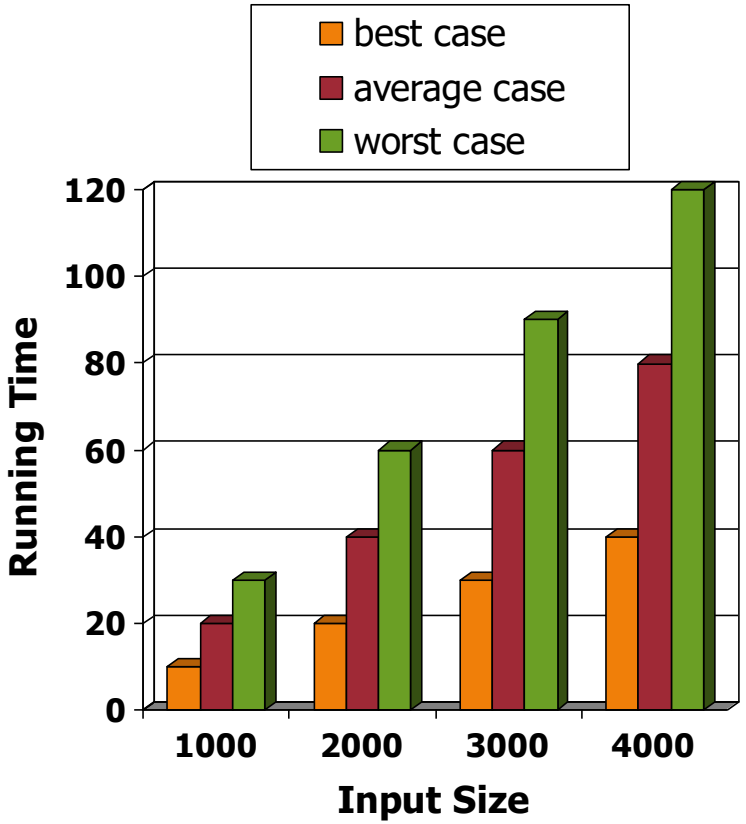
Algorithm *contains* (*A*, *e*)
  Input array *A* of *n* integers
  Output true if *e* exists in *A*,
    false otherwise

  for *i* ← 0 **to** *n* – 1 **do**
    if *A*[*i*] = *e* **then**
      return *true*
  return *false*

```
// Java
boolean contains (int A[], int e)
{
  for (int i = 0; i < A.length, i++)
    if (A[i] == e)
      return true;
  return false;
}
```

Worst Case

- We generally focus on the worst-case running time
- Easier to analyze
- We can estimate the maximum time for the algorithm to finish, which is crucial to applications such as games, finance and robotics

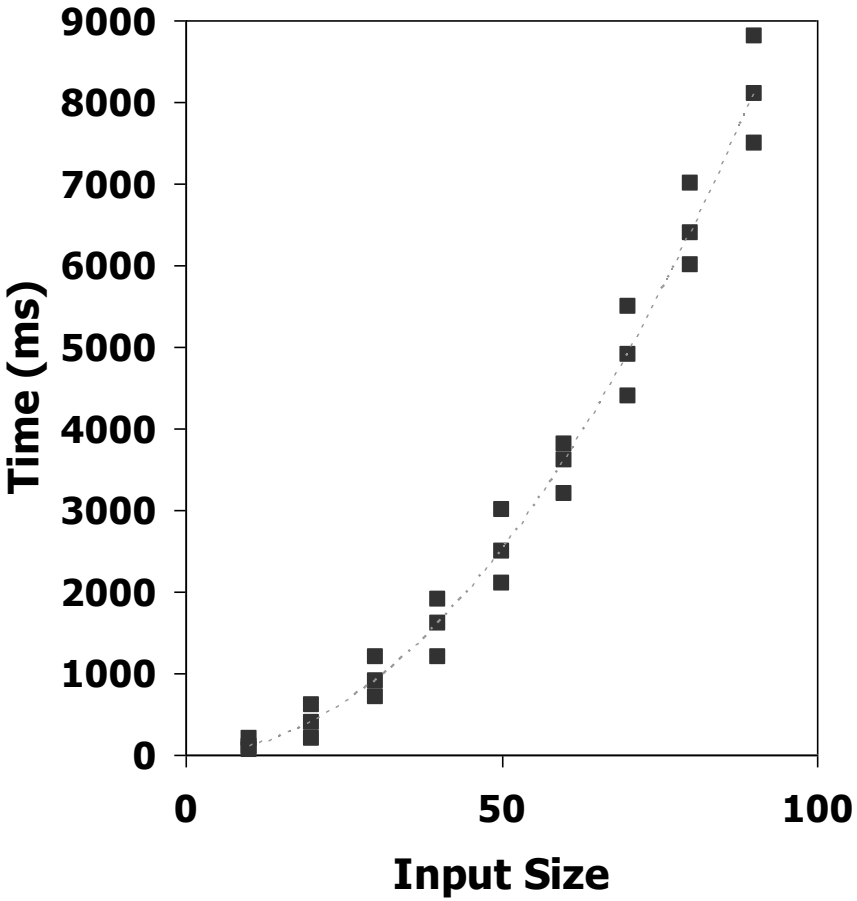


Running Time Analysis

- 2 techniques:
  - Experimental Studies
  - Theoretical Analysis

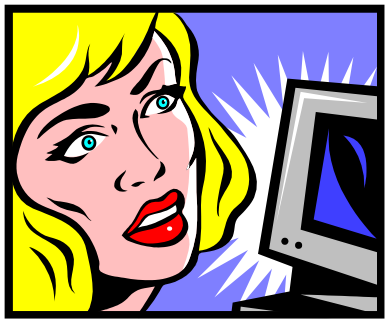
Experimental Studies

1. Write a program implementing the algorithm
2. Run the program with inputs of varying size and composition
3. Use programming language's time method to get an accurate measure of the actual running time
4. Plot the results



Limitations of Experiments

- No hardware to process huge amount of inputs
- Difficult to find complete or sufficient inputs to produce indicative results
- Difficult to find the same hardware and software to compare algorithms in a fair manner



Theoretical Analysis

- Uses pseudocode instead of an implementation
- Characterizes running time as a function of the input size, *n*.
- Advantages:
  - Takes into account all possible inputs
  - Allows us to evaluate the speed of an algorithm independent of the hardware/software

Pseudocode

- High-level description of an algorithm
- Less detailed than a programming language
- More than one way to write a pseudocode code
- Example: find max element of an array

Algorithm *arrayMax*(*A*, *n*)
  Input array *A* of *n* integers
  Output maximum element of *A*

  currentMax ← *A*[0]
  for *i* ← 1 **to** *n* − 1 **do**
    if *A*[*i*] > *currentMax* **then**
      currentMax ← *A*[*i*]
  return *currentMax*

```
Algorithm arrayMax(A, n)
  Input: array A of n integers
  Output: maximum element of A

  currentMax = A[0]
  for i = 1 to n - 1
    if A[i] > currentMax
      currentMax = A[i]
  return currentMax
```

Primitive Operations

- Basic computations performed by an algorithm
- Identifiable in pseudocode
- Largely independent from the programming language
- Exact definition not important (we will see why later)
- Assumed to take a constant amount of time

- Examples:
  - Performing an arithmetic ops
  - Assigning a value to a variable
  - Indexing into an array
  - Calling a method
  - Returning from a method
  - Comparing 2 variables

Counting Primitive Operations

- We determine the maximum number of primitive operations executed by an algorithm (worst case), as a function of the input size
- This way of counting primitive operations in detail is for illustration only. Don't count in this way

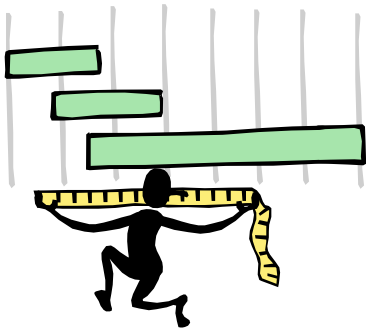
Algorithm *arrayMax*(A, n)	# operations
currentMax $\leftarrow A[0]$	2
for $i \leftarrow 1$ **to** $n - 1$ **do**	$1 + n$
if $A[i] > currentMax$ **then**	$2(n - 1)$
$currentMax \leftarrow A[i]$	$2(n - 1)$
{ increment counter i }	$2(n - 1)$
return *currentMax*	1
	Total = $7n - 2$

Counting Primitive Operations (cont.)

- C++/Java version of *arrayMax* algorithm

```
int arrayMax (int A[], int n) {        # operations
  int currentMax = A[0];                    2
  for (int i=1;                             1
       i < n;                               n
       i++)                              2(n – 1)
    if (A[i] > currentMax)               2(n – 1)
      currentMax = A[i];                 2(n – 1)
  return currentMax                         1
}                                  Total = 7n – 2
```

Estimating Running Time



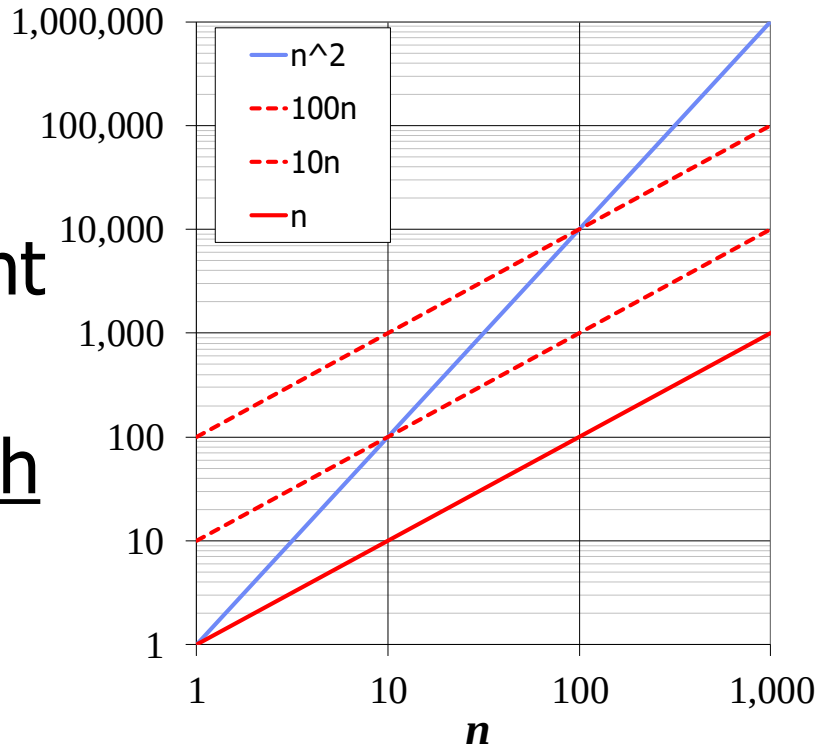
- In the worst case, ***arrayMax*** executes $7\mathbf{n} - 2$ primitive operations whereby the 'if' loop is always true.
- In the best case, ***arrayMax*** executes $5\boldsymbol{n}$ primitive operations whereby the 'if' loop is always false.
- Let $\boldsymbol{T}(\boldsymbol{n})$ be worst-case time of ***arrayMax.*** Then

$$\boldsymbol{T}(\boldsymbol{n}) = 7\boldsymbol{n} - 2$$

- Next, we will see why we don't have to count in detail the primitive operations.

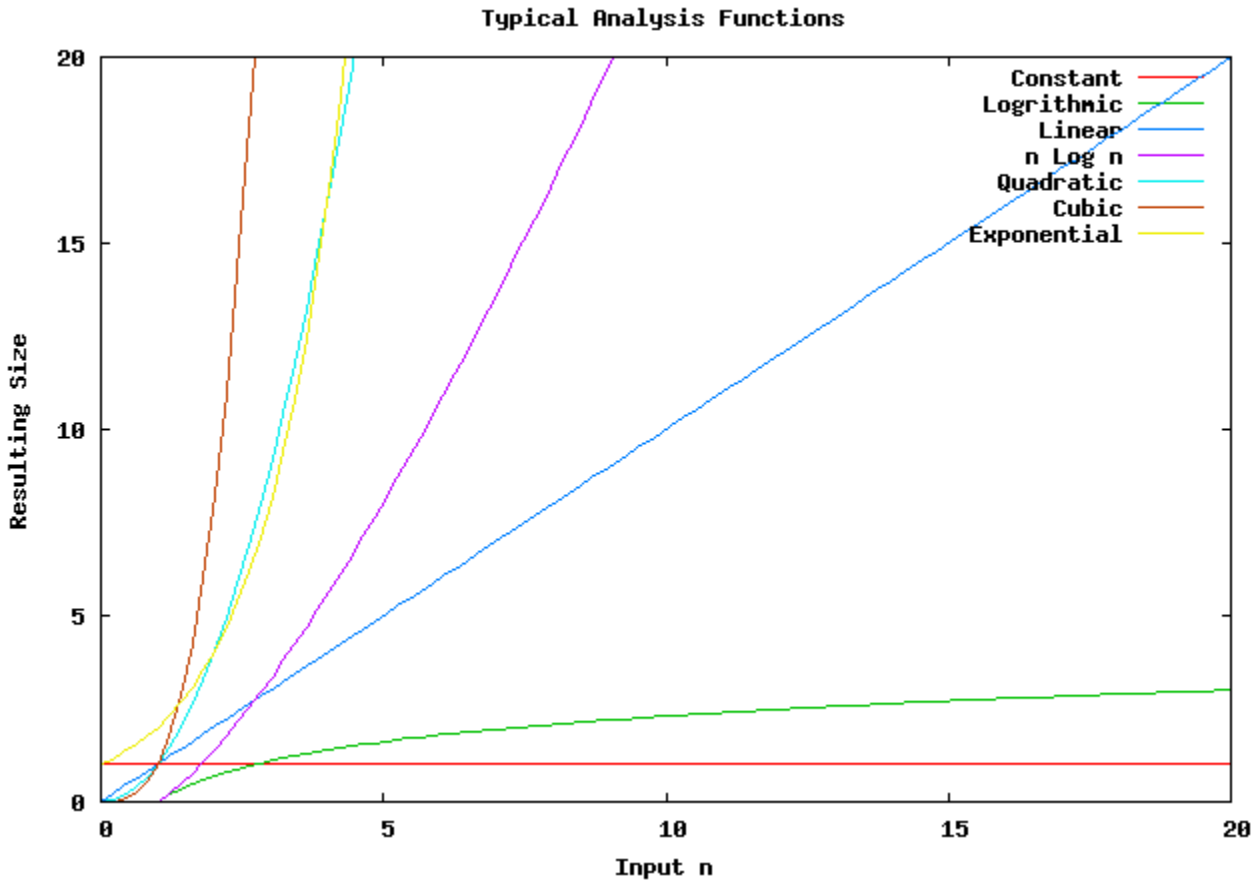
Growth Rate of Running Time

- Fact: Changing the hardware/software environment
  - Affects $T(n)$ by a constant factor, but
  - Does not alter the growth rate of $T(n)$
    - n, 10n and 100n grow at the same rate



Growth Rates of Functions

- From lowest to highest:
  - Constant ≈ 1
  - Logarithmic ≈ log n
  - Linear ≈ n
  - N-Log-N ≈ n log n
  - Quadratic ≈ n^2
  - Cubic ≈ n^3
  - Exponential ≈ 2^n

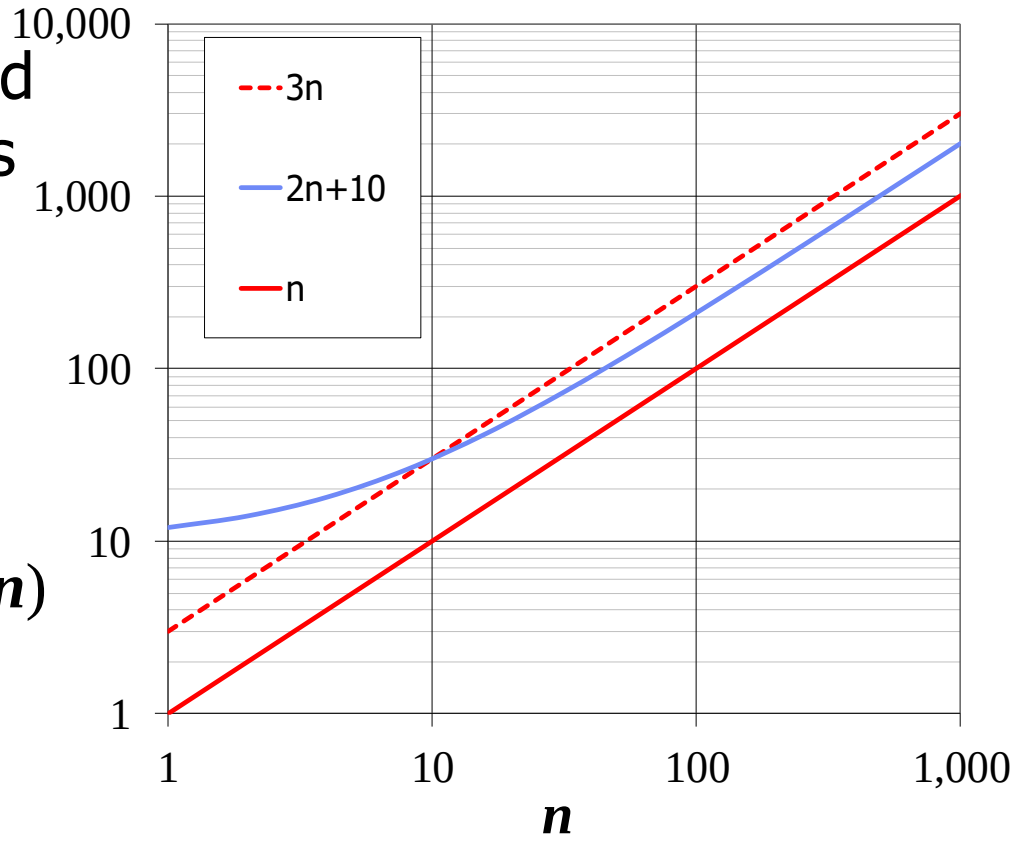


Big-Oh Notation (§1.2)

- Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that

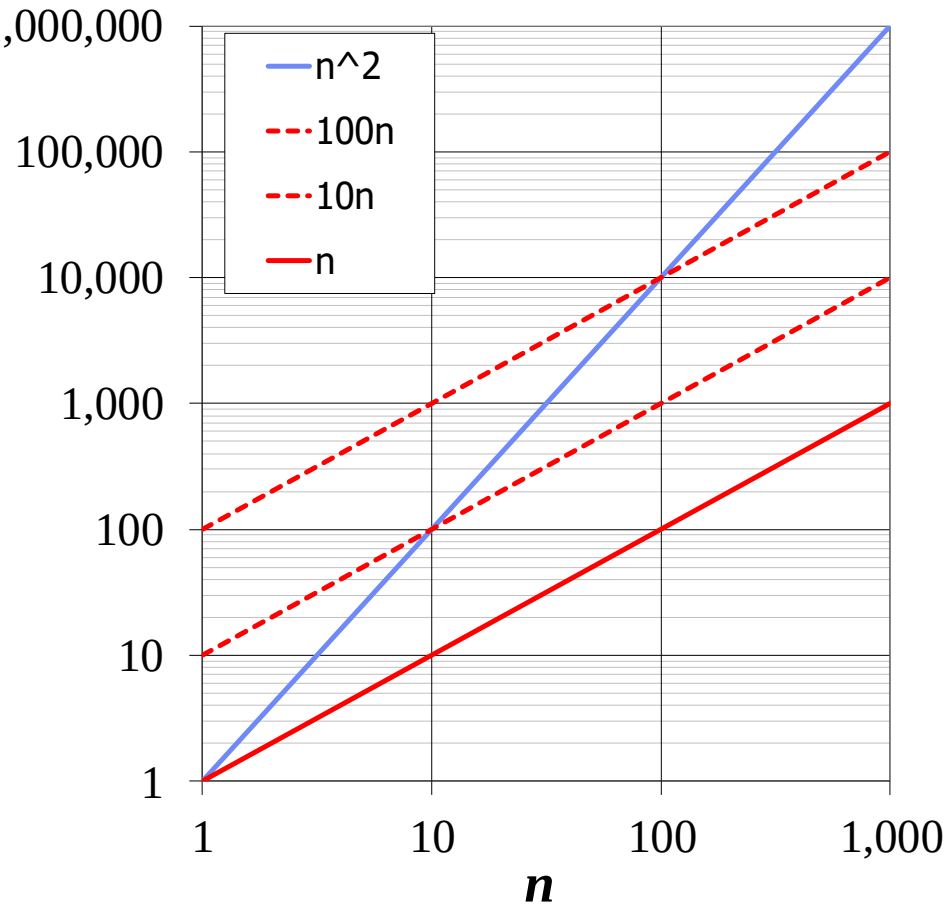
  $f(n) \leq cg(n)$ for $n \geq n_0$

- Example: $2n + 10$ is $O(n)$
  - $2n + 10 \leq cn$
  - $(c - 2)\, n \geq 10$
  - $n \geq 10/(c - 2)$
  - Pick $c = 3$ and $n_0 = 10$



Big-Oh Example

- Example: n^2 is not $O(n)$
  - $n^2 \leq cn$
  - $n \leq c$
  - The above inequality cannot be satisfied since c must be a constant



Big-Oh and Growth Rate

- The big-Oh notation gives an upper bound on the growth rate of a function
- The statement "$f(n)$ is $O(g(n))$" means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Big-Oh Rules

- If is $f(n)$ a polynomial of degree d, then $f(n)$ is $O(n^d)$, i.e.,
  1. Drop lower-order terms
  2. Drop constant factors
- Use the smallest possible class of functions
  - Say "$2n$ is $O(n)$" instead of "$2n$ is $O(n^2)$"
- Use the simplest expression of the class
  - Say "$3n + 5$ is $O(n)$" instead of "$3n + 5$ is $O(3n)$"

Asymptotic Algorithm Analysis

- Determines running time in big-Oh notation
- To perform the asymptotic analysis
  1. We find the worst-case number of primitive operations executed as a function of the input size
  2. We express this function with big-Oh notation
- Example:
  - We determine that algorithm ***arrayMax*** executes at most $7\boldsymbol{n} - 2$ primitive operations
  - We say that algorithm ***arrayMax*** "runs in $\boldsymbol{O(n)}$ time"
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Asymptotic Algorithm Analysis

- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Algorithm *arrayMax*(*A*, *n*)	# operations
currentMax ← *A*[0]	**1**
for *i* ← 1 **to** *n* – 1 **do**	***n***
if *A*[*i*] > *currentMax* **then**	***n***
currentMax ← *A*[*i*]	***n***
{ increment counter ***i*** }	***n***
return *currentMax*	**1**
	Total = ***O***(***n***)

The Importance of Asymptotics

- An algorithm with an asymptotically fast running time (low growth rate) can process more inputs than an algorithm with an asymptotically slow running time (high growth rate)

Maximum Problem Size (n)

Running Time	1 second	1 minute	1 hour
O(n)	2,500	150,000	9,000,000
O(nlog n)	4,096	166,666	7,826,087
$O(n^{2})$	707	5,477	42,426
$O(n^{4})$	31	88	244
$O(2^{n})$	19	25	31

Computing Prefix Sums

- Another example: prefix sums
- The i-th prefix sum of an array X is sum of the first $(i + 1)$ elements of X:

$$A[i] = (X[0] + X[1] + \ldots + X[i])$$

- We will compare 2 algorithms with different running time for computing prefix sums
- prefixSum1 runs in $O(n^2)$
- prefixSum2 runs in $O(n)$
- Which one runs faster?

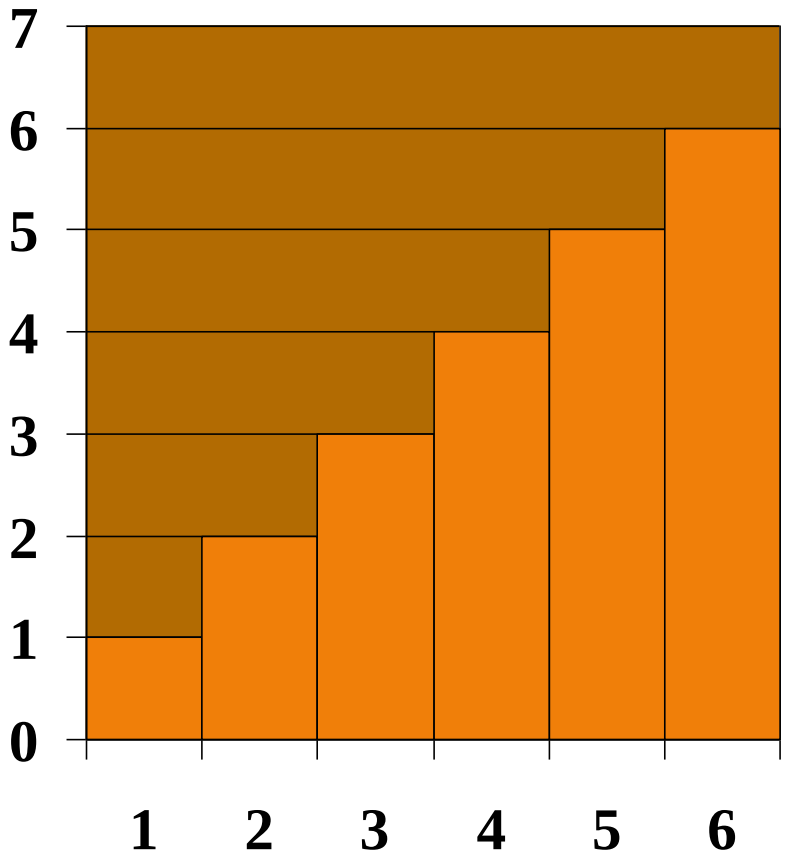
PrefixSums1 (Quadratic)

- The following algorithm computes prefix averages in quadratic time n^2

Algorithm *prefixSum1* (X, n)	
Input array X of n integers	
Output array A of prefix sums of X	#operations
$A \leftarrow$ new array of n integers	1
for $i \leftarrow 0$ **to** $n-1$ **do**	n
$s \leftarrow X[0]$	n
for $j \leftarrow 1$ **to** i **do**	$1+2+\ldots+(n-1) \approx n^2$
$s \leftarrow s + X[j]$	$1+2+\ldots+(n-1) \approx n^2$
$A[i] \leftarrow s$	n
return A	1
	$T(n) = O(n^2)$

Arithmetic Progression

- The sum of the first n integers $(1 + 2 + \ldots + n)$ is $n(n + 1) / 2$
  - There is a simple visual proof of this fact
- Thus, algorithm ***prefixSum1*** runs in $O(n^2)$ time

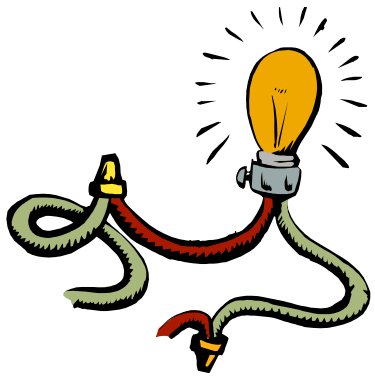


PrefixSum2 (Linear)

- The following algorithm computes prefix sums in linear time *n* by using the previous sum

```
Algorithm prefixSums2 (X, n)
    Input array X of n integers
    Output array A of prefix averages of X          #operations
    A ← new array of n integers                          1
    s ← 0                                                1
    for i ← 0 to n – 1 do                                n
        s ← s + X[i]                                     n
        A[i] ← s                                         n
    return A                                             1
                                                  T(n) = O(n)
```

Intuition for Asymptotic Notation



Big-Oh

- f(n) is O(g(n)) if f(n) is asymptotically **less than or equal** to g(n)

big-Omega

- f(n) is Ω(g(n)) if f(n) is asymptotically **greater than or equal** to g(n)

big-Theta

- f(n) is Θ(g(n)) if f(n) is asymptotically **equal** to g(n)

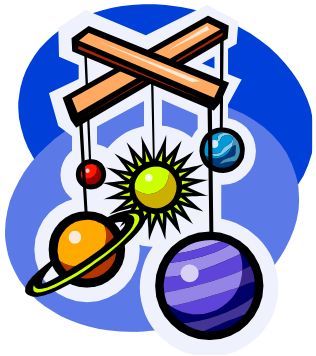
little-oh

- f(n) is o(g(n)) if f(n) is asymptotically **strictly less** than g(n)

little-omega

- f(n) is ω(g(n)) if is asymptotically **strictly greater** than g(n)

Example Uses of the Relatives of Big-Oh



- $5n^2$ is $\Omega(n^2)$
- $5n^2$ is $\Omega(n)$
- $5n^2$ is $\omega(n)$

Time Complexity, $T(n)$

- Time complexity, $T(n)$, refers to the use of asymptotic notation (O, Ω, Θ, o, ω) in denoting running time
- If 2 algorithms accomplish the same task with different time complexities:
  - One is faster than another
  - As input *n* increases further, the performance difference becomes more obvious
- Faster algorithm is generally preferred

Time Complexity Comparison

- A fast algorithm has a low time complexity, while a slow algorithm has a high time complexity.
- Ranking of selected time complexities:
  - Constant ≈ 1 (lowest $T(n)$, fastest speed)
  - Logarithmic $\approx \log n$
  - Linear $\approx n$
  - N-Log-N $\approx n \log n$
  - Quadratic $\approx n^2$
  - Cubic $\approx n^3$
  - Exponential $\approx 2^n$ (highest $T(n)$, slowest speed)